Nonlinear Simulation of an Inverted Pendulum on a Cart

J.C. Vaught

Objective

The objective of this assignment is to model the nonlinear motion of an unforced inverted pendulum on a freely translating cart and to compare two independent implementations. The first implementation evaluates the equations of motion directly in Simulink. The second represents the same mechanism with physical components in Simscape Multibody. The cart position $x(t)$ and pendulum angle $\theta(t)$ are compared after releasing the pendulum five degrees from the unstable upright equilibrium with zero initial velocity.

Methodology

The pendulum is treated as a point mass m at distance ℓ from a frictionless cart of mass M. The coordinate x is positive to the right, and θ is measured clockwise from upright. A viscous rotational damper with coefficient c_θ acts at the pivot. Lagrange's equations give the coupled nonlinear model

$$(M+m)\ddot{x} + m\ell\cos(\theta)\ddot{\theta} - m\ell\sin(\theta)\dot{\theta}^2 = 0,$$

and

$$m\ell\cos(\theta)\ddot{x} + m\ell^2\ddot{\theta} + c_\theta\dot{\theta} - mg\ell\sin(\theta) = 0.$$

At each Simulink time step, the two equations are assembled as a 2×2 mass-matrix system and solved for $\ddot{x}$ and $\ddot{\theta}$. Two integrator chains recover $x(t)$ and $\theta(t)$. The second model uses a world frame, a prismatic cart joint, a damped revolute pivot, a rigid transform of length ℓ, and a concentrated spherical mass. Both models use $M = 2.0$ kg, $m = 0.5$ kg, $\ell = 1.0$ m, $c_\theta = 0.01\ N \cdot m \cdot \frac{s}{\text{rad}}$, $g = 9.81\ \frac{m}{s^2}$, and $\theta(0) = 5°$. Variable-step solvers run for 20 s with a maximum step of 0.01 s.

The two responses are interpolated onto a common 0.01 s time grid. Agreement is assessed from the maximum absolute difference in cart position and pendulum angle over the simulation interval.

Results and Discussion

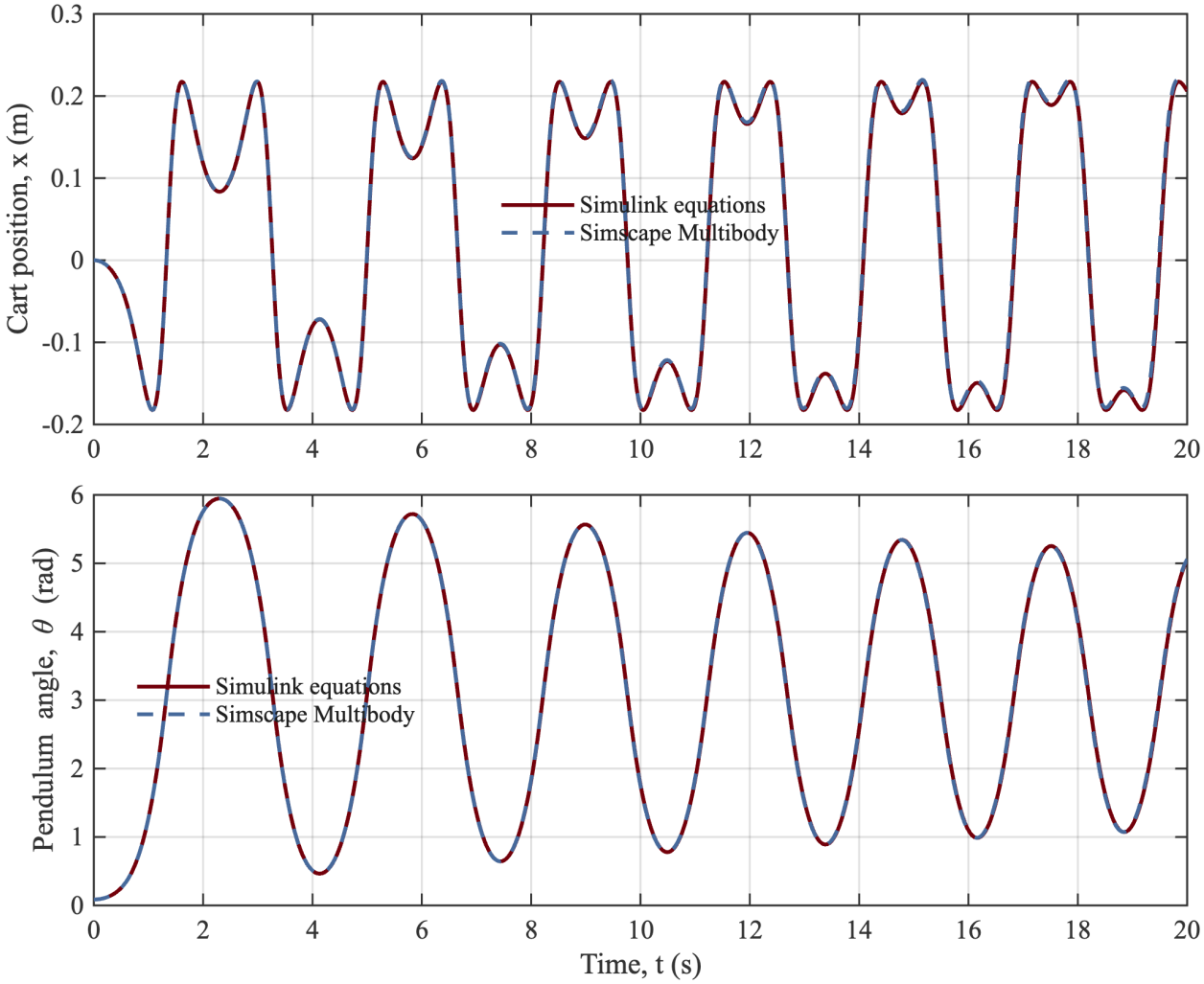

Figure 1: Cart position and pendulum angle from the direct-equations and physical-component models.

The five-degree perturbation grows immediately because the upright equilibrium is unstable. The pendulum falls through the downward equilibrium and continues into a damped oscillation about $\theta = \pi$ rad. Over 20 s, the direct model predicts $-0.183 \leq x \leq 0.217$ m and $0.087 \leq \theta \leq 5.947$ rad. The cart remains bounded because there is no external horizontal force, while pivot damping gradually removes mechanical energy and contracts the angular oscillation about the downward equilibrium.

The two independently constructed responses nearly overlap throughout the simulation. Their maximum absolute differences are 0.014 m in x and 0.056 rad in θ. The small phase separation late in the record is expected because the release begins near an unstable equilibrium, where small numerical and geometric differences amplify before damping dominates. The agreement supports both the nonlinear equation implementation and the orientation, gravity, damping, and sensing choices in the Multibody model.

Appendix A. Direct-Equations Simulink Model

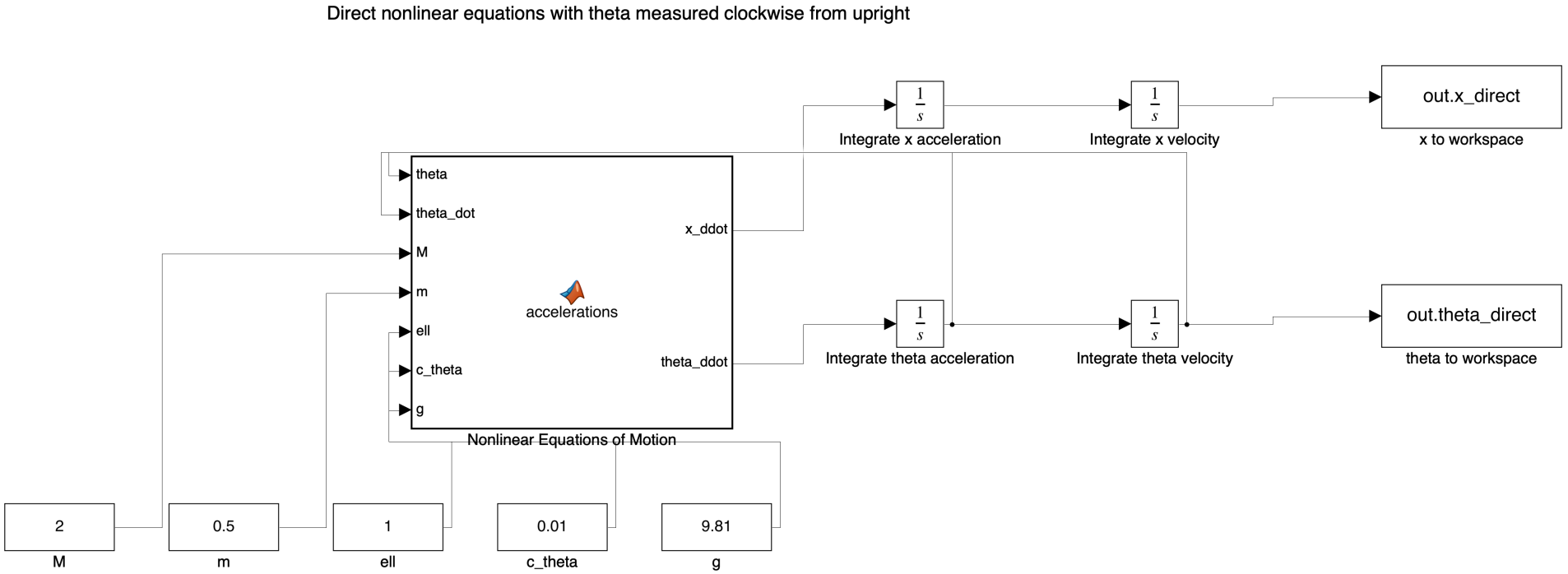

Figure 2: Editable Simulink implementation of the nonlinear mass-matrix equations.

Appendix B. Simscape Multibody Model

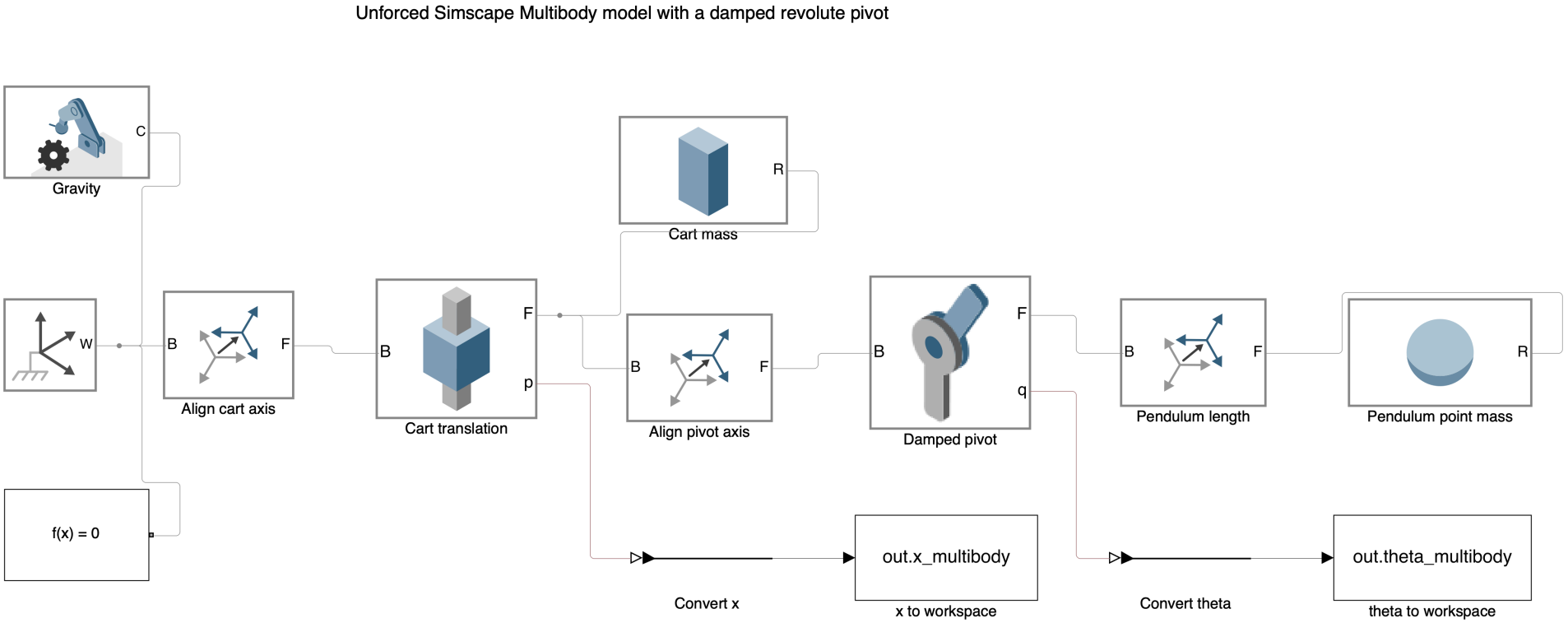

Figure 3: Editable physical-component model with cart translation, gravity, damped pivot, rigid pendulum length, and point mass.

Appendix C. Reproducible MATLAB Build and Simulation Code

```
%% Mini-Project 1. Nonlinear inverted pendulum on a translating cart.
% J.C. Vaught
%
% This script builds two editable models of the same unforced system. The
% first model implements the nonlinear equations of motion directly in
% Simulink. The second model uses physical components from Simscape
% Multibody. Both models start five degrees from the unstable upright
% equilibrium with zero translational and angular velocity.

clear;
close all;
clc;

project_dir = fileparts(mfilename("fullpath"));
old_dir = pwd;
cleanup_dir = onCleanup(@() cd(old_dir));
cd(project_dir);

%% Physical parameters.
M = 2.0;              % Cart mass, kg.
m = 0.5;              % Pendulum point mass, kg.
ell = 1.0;            % Pivot-to-mass distance, m.
c_theta = 0.01;       % Pivot damping, N*m*s/rad.
g = 9.81;             % Gravitational acceleration, m/s^2.
theta0 = deg2rad(5);  % Initial clockwise perturbation from upright, rad.
t_stop = 20.0;        % Simulation duration, s.

parameters = struct( ...
    "M", M, ...
    "m", m, ...
    "ell", ell, ...
    "c_theta", c_theta, ...
    "g", g, ...
    "theta0", theta0, ...
    "t_stop", t_stop);

%% Build, simulate, and save the direct-equations model.
direct_model = "inverted_pendulum_simulink";
build_direct_model(direct_model, parameters);
direct_output = sim(direct_model);
x_direct = direct_output.x_direct;
theta_direct = direct_output.theta_direct;

%% Build, simulate, and save the Simscape Multibody model.
multibody_model = "inverted_pendulum_multibody";
build_multibody_model(multibody_model, parameters);
multibody_output = sim(multibody_model);
x_multibody = multibody_output.x_multibody;
theta_multibody = multibody_output.theta_multibody;

%% Interpolate both responses onto a common time base for comparison.
t = (0:0.01:t_stop).';
x_direct_common = interp1( ...
    x_direct.Time, squeeze(x_direct.Data), t, "pchip");
theta_direct_common = interp1( ...
    theta_direct.Time, squeeze(theta_direct.Data), t, "pchip");
x_multibody_common = interp1( ...
```

```
    x_multibody.Time, squeeze(x_multibody.Data), t, "pchip");
theta_multibody_common = interp1( ...
    theta_multibody.Time, squeeze(theta_multibody.Data), t, "pchip");
x_direct_common = x_direct_common(:);
theta_direct_common = theta_direct_common(:);
x_multibody_common = x_multibody_common(:);
theta_multibody_common = theta_multibody_common(:);

results = table( ...
    t, ...
    x_direct_common, ...
    theta_direct_common, ...
    x_multibody_common, ...
    theta_multibody_common, ...
    'VariableNames', { ...
        'time_s', ...
        'x_simulink_m', ...
        'theta_simulink_rad', ...
        'x_multibody_m', ...
        'theta_multibody_rad'});
writetable(results, "simulation_results.csv");
save( ...
    "simulation_results.mat", ...
    "parameters", ...
    "x_direct", ...
    "theta_direct", ...
    "x_multibody", ...
    "theta_multibody", ...
    "results");

%% Plot the required responses using course and project style rules.
garnet = [115, 0, 10] / 255;
atlantic = [70, 106, 159] / 255;
neutral = [54, 54, 54] / 255;

response_figure = figure( ...
    "Color", "w", ...
    "Units", "inches", ...
    "Position", [1, 1, 7.25, 5.8], ...
    "Name", "Mini-Project 1 Responses");
layout = tiledlayout(response_figure, 2, 1, ...
    "TileSpacing", "compact", ...
    "Padding", "compact");

position_axes = nexttile(layout);
plot(position_axes, t, x_direct_common, ...
    "Color", garnet, "LineStyle", "-", "LineWidth", 1.6);
hold(position_axes, "on");
plot(position_axes, t, x_multibody_common, ...
    "Color", atlantic, "LineStyle", "--", "LineWidth", 1.6);
hold(position_axes, "off");
ylabel(position_axes, "Cart position, x (m)");
legend(position_axes, {"Simulink equations", "Simscape Multibody"}, ...
    "Location", "best", "Box", "off");
style_axes(position_axes, neutral);

angle_axes = nexttile(layout);
plot(angle_axes, t, theta_direct_common, ...
    "Color", garnet, "LineStyle", "-", "LineWidth", 1.6);
```

```
hold(angle_axes, "on");
plot(angle_axes, t, theta_multibody_common, ...
    "Color", atlantic, "LineStyle", "--", "LineWidth", 1.6);
hold(angle_axes, "off");
xlabel(angle_axes, "Time, t (s)");
ylabel(angle_axes, "Pendulum angle, \theta (rad)");
legend(angle_axes, {"Simulink equations", "Simscape Multibody"}, ...
    "Location", "best", "Box", "off");
style_axes(angle_axes, neutral);

exportgraphics(response_figure, "response_comparison.png", ...
    "Resolution", 300);
exportgraphics(response_figure, "response_comparison.pdf", ...
    "ContentType", "vector");

%% Quantify agreement between the two independent implementations.
x_error = max(abs(x_direct_common - x_multibody_common));
theta_error = max(abs(theta_direct_common - theta_multibody_common));
fprintf("Maximum cart-position difference: %.6e m\n", x_error);
fprintf("Maximum pendulum-angle difference: %.6e rad\n", theta_error);

%% Export clean model diagrams for the report appendix.
export_model_diagram(direct_model, "simulink_model.png");
export_model_diagram(multibody_model, "multibody_model.png");

close_system(direct_model, false);
close_system(multibody_model, false);

%% Local functions.
function build_direct_model(model_name, p)
    close_if_loaded(model_name);
    delete_if_present(model_name + ".slx");
    new_system(model_name);
    open_system(model_name);

    set_param(model_name, ...
        "Solver", "ode45", ...
        "StopTime", num2str(p.t_stop), ...
        "MaxStep", "0.01", ...
        "ReturnWorkspaceOutputs", "on");

    equation_block = model_name + "/Nonlinear Equations of Motion";
    add_block( ...
        "simulink/User-Defined Functions/MATLAB Function", ...
        equation_block, ...
        "Position", [300, 145, 505, 315]);
    chart = find( ...
        sfroot, ...
        "-isa", "Stateflow.EMChart", ...
        "Path", char(equation_block));
    chart.Script = strjoin([ ...
        "function [x_ddot, theta_ddot] = accelerations(theta, theta_dot, M, m, ell, c_theta, g)" ...
        "% Solve the coupled nonlinear mass-matrix equations." ...
        "s = sin(theta);" ...
        "c = cos(theta);" ...
        "mass_matrix = [M + m, m * ell * c; m * ell * c, m * ell^2];" ...
        "forcing = [m * ell * s * theta_dot^2; m * g * ell * s - c_theta * theta_dot];" ...
        "acceleration = mass_matrix \ forcing;" ...
        "x_ddot = acceleration(1);" ...
```

```
        "theta_ddot = acceleration(2);" ...
        "end"], newline);

    x_velocity = model_name + "/Integrate x acceleration";
    x_position = model_name + "/Integrate x velocity";
    theta_velocity = model_name + "/Integrate theta acceleration";
    theta_position = model_name + "/Integrate theta velocity";
    add_block("simulink/Continuous/Integrator", x_velocity, ...
        "Position", [610, 105, 640, 135], "InitialCondition", "0");
    add_block("simulink/Continuous/Integrator", x_position, ...
        "Position", [760, 105, 790, 135], "InitialCondition", "0");
    add_block("simulink/Continuous/Integrator", theta_velocity, ...
        "Position", [610, 245, 640, 275], "InitialCondition", "0");
    add_block("simulink/Continuous/Integrator", theta_position, ...
        "Position", [760, 245, 790, 275], ...
        "InitialCondition", num2str(p.theta0, 17));

    constant_names = ["M", "m", "ell", "c_theta", "g"];
    constant_values = [p.M, p.m, p.ell, p.c_theta, p.g];
    constant_x = [40, 145, 250, 355, 460];
    for index = 1:numel(constant_names)
        add_block( ...
            "simulink/Sources/Constant", ...
            model_name + "/" + constant_names(index), ...
            "Position", [constant_x(index), 375, ...
                constant_x(index) + 70, 405], ...
            "Value", num2str(constant_values(index), "%.6g"));
    end

    x_sink = model_name + "/x to workspace";
    theta_sink = model_name + "/theta to workspace";
    add_block("simulink/Sinks/To Workspace", x_sink, ...
        "Position", [920, 95, 1035, 135], ...
        "VariableName", "x_direct", ...
        "SaveFormat", "Timeseries");
    add_block("simulink/Sinks/To Workspace", theta_sink, ...
        "Position", [920, 235, 1035, 275], ...
        "VariableName", "theta_direct", ...
        "SaveFormat", "Timeseries");

    add_line(model_name, "Nonlinear Equations of Motion/1", ...
        "Integrate x acceleration/1", "autorouting", "on");
    add_line(model_name, "Integrate x acceleration/1", ...
        "Integrate x velocity/1", "autorouting", "on");
    add_line(model_name, "Integrate x velocity/1", ...
        "x to workspace/1", "autorouting", "on");
    add_line(model_name, "Nonlinear Equations of Motion/2", ...
        "Integrate theta acceleration/1", "autorouting", "on");
    add_line(model_name, "Integrate theta acceleration/1", ...
        "Integrate theta velocity/1", "autorouting", "on");
    add_line(model_name, "Integrate theta velocity/1", ...
        "theta to workspace/1", "autorouting", "on");

    add_line(model_name, "Integrate theta velocity/1", ...
        "Nonlinear Equations of Motion/1", "autorouting", "on");
    add_line(model_name, "Integrate theta acceleration/1", ...
        "Nonlinear Equations of Motion/2", "autorouting", "on");
    for index = 1:numel(constant_names)
        add_line( ...
```

```
            model_name, ...
            constant_names(index) + "/1", ...
            "Nonlinear Equations of Motion/" + string(index + 2), ...
            "autorouting", "on");
    end

    add_annotation(model_name, ...
        "Direct nonlinear equations with theta measured clockwise from upright", ...
        [245, 55, 735, 85]);
    save_system(model_name);
end

function build_multibody_model(model_name, p)
    close_if_loaded(model_name);
    delete_if_present(model_name + ".slx");
    load_system("sm_lib");
    load_system("nesl_utility");
    new_system(model_name);
    open_system(model_name);

    set_param(model_name, ...
        "Solver", "ode23t", ...
        "StopTime", num2str(p.t_stop), ...
        "MaxStep", "0.01", ...
        "ReturnWorkspaceOutputs", "on");

    world = add_named_library_block( ...
        "sm_lib", "World Frame", model_name + "/World Frame", ...
        [40, 220, 100, 280]);
    mechanism = add_named_library_block( ...
        "sm_lib", "Mechanism Configuration", ...
        model_name + "/Gravity", [40, 80, 135, 140]);
    set_param(mechanism, "GravityVector", "[0 0 -9.81]");
    solver = add_named_library_block( ...
        "nesl_utility", "Solver Configuration", ...
        model_name + "/Solver Configuration", [40, 345, 135, 405]);

    axis_alignment = add_named_library_block( ...
        "sm_lib", "Rigid Transform", model_name + "/Align cart axis", ...
        [145, 215, 230, 285]);
    set_param(axis_alignment, ...
        "RotationMethod", "StandardAxis", ...
        "RotationStandardAxis", "+Y", ...
        "RotationAngle", "90", ...
        "RotationAngleUnits", "deg");

    prismatic = add_named_library_block( ...
        "sm_lib", "Prismatic Joint", model_name + "/Cart translation", ...
        [285, 205, 390, 295]);
    set_param(prismatic, "SensePosition", "on");

    cart = add_named_library_block( ...
        "sm_lib", "Brick Solid", model_name + "/Cart mass", ...
        [445, 100, 555, 170]);
    set_param(cart, ...
        "BrickDimensions", "[0.2 0.3 0.6]", ...
        "BasedOnType", "Mass", ...
        "Mass", num2str(p.M, 17), ...
        "GraphicDiffuseColor", "[0.45 0.45 0.45]");
```

```
pivot_axis = add_named_library_block( ...
    "sm_lib", "Rigid Transform", model_name + "/Align pivot axis", ...
    [450, 230, 545, 300]);
set_param(pivot_axis, ...
    "RotationMethod", "StandardAxis", ...
    "RotationStandardAxis", "+X", ...
    "RotationAngle", "-90", ...
    "RotationAngleUnits", "deg");

revolute = add_named_library_block( ...
    "sm_lib", "Revolute Joint", model_name + "/Damped pivot", ...
    [610, 205, 715, 305]);
set_param(revolute, ...
    "PositionTargetSpecify", "on", ...
    "PositionTargetPriority", "High", ...
    "PositionTargetValue", num2str(p.theta0, 17), ...
    "PositionTargetValueUnits", "rad", ...
    "DampingCoefficient", num2str(p.c_theta, 17), ...
    "DampingCoefficientUnits", "N*m/(rad/s)", ...
    "SensePosition", "on");

bob_offset = add_named_library_block( ...
    "sm_lib", "Rigid Transform", model_name + "/Pendulum length", ...
    [775, 220, 870, 290]);
set_param(bob_offset, ...
    "TranslationMethod", "Cartesian", ...
    "TranslationCartesianOffset", sprintf("[-%.17g 0 0]", p.ell), ...
    "TranslationCartesianOffsetUnits", "m");

bob = add_named_library_block( ...
    "sm_lib", "Spherical Solid", model_name + "/Pendulum point mass", ...
    [925, 220, 1045, 290]);
set_param(bob, ...
    "SphereRadius", "0.06", ...
    "InertiaType", "Custom", ...
    "Mass", num2str(p.m, 17), ...
    "CenterOfMass", "[0 0 0]", ...
    "MomentsOfInertia", "[1e-9 1e-9 1e-9]", ...
    "ProductsOfInertia", "[0 0 0]", ...
    "GraphicDiffuseColor", "[0.45 0 0.04]");

connect_physical(model_name, world, "RConn", mechanism, "RConn");
connect_physical(model_name, world, "RConn", solver, "RConn");
connect_physical(model_name, world, "RConn", axis_alignment, "LConn");
connect_physical(model_name, axis_alignment, "RConn", prismatic, "LConn");
connect_physical(model_name, prismatic, "RConn", cart, "RConn");
connect_physical(model_name, prismatic, "RConn", pivot_axis, "LConn");
connect_physical(model_name, pivot_axis, "RConn", revolute, "LConn");
connect_physical(model_name, revolute, "RConn", bob_offset, "LConn");
connect_physical(model_name, bob_offset, "RConn", bob, "RConn");

converter_source = find_named_library_block( ...
    "nesl_utility", "PS-Simulink Converter");
x_converter = model_name + "/Convert x";
theta_converter = model_name + "/Convert theta";
add_block(converter_source, x_converter, ...
    "Position", [460, 365, 545, 410]);
add_block(converter_source, theta_converter, ...
```

```
        "Position", [775, 365, 860, 410]);
    set_converter_unit(x_converter, "m");
    set_converter_unit(theta_converter, "rad");

    x_sink = model_name + "/x to workspace";
    theta_sink = model_name + "/theta to workspace";
    add_block("simulink/Sinks/To Workspace", x_sink, ...
        "Position", [600, 360, 720, 415], ...
        "VariableName", "x_multibody", ...
        "SaveFormat", "Timeseries");
    add_block("simulink/Sinks/To Workspace", theta_sink, ...
        "Position", [915, 360, 1045, 415], ...
        "VariableName", "theta_multibody", ...
        "SaveFormat", "Timeseries");

    prismatic_ports = get_param(prismatic, "PortHandles");
    revolute_ports = get_param(revolute, "PortHandles");
    x_converter_ports = get_param(x_converter, "PortHandles");
    theta_converter_ports = get_param(theta_converter, "PortHandles");
    add_line(model_name, prismatic_ports.RConn(2), ...
        x_converter_ports.LConn(1), "autorouting", "on");
    add_line(model_name, revolute_ports.RConn(2), ...
        theta_converter_ports.LConn(1), "autorouting", "on");
    add_line(model_name, "Convert x/1", "x to workspace/1", ...
        "autorouting", "on");
    add_line(model_name, "Convert theta/1", "theta to workspace/1", ...
        "autorouting", "on");

    add_annotation(model_name, ...
        "Unforced Simscape Multibody model with a damped revolute pivot", ...
        [270, 25, 800, 60]);
    save_system(model_name);
end

function block_path = add_named_library_block( ...
        library_name, block_name, destination, position)
    source = find_named_library_block(library_name, block_name);
    add_block(source, destination, "Position", position);
    block_path = destination;
end

function source = find_named_library_block(library_name, block_name)
    load_system(library_name);
    candidates = find_system( ...
        library_name, ...
        "LookUnderMasks", "all", ...
        "FollowLinks", "on", ...
        "Name", block_name);
    if isempty(candidates)
        error("Block '%s' was not found in '%s'.", block_name, library_name);
    end
    source = candidates{1};
end

function connect_physical(model_name, block_a, field_a, block_b, field_b)
    ports_a = get_param(block_a, "PortHandles");
    ports_b = get_param(block_b, "PortHandles");
    add_line( ...
        model_name, ...
```

```
        ports_a.(field_a)(1), ...
        ports_b.(field_b)(1), ...
        "autorouting", "on");
end

function set_converter_unit(block_path, unit)
    parameters = get_param(block_path, "DialogParameters");
    if isfield(parameters, "Unit")
        set_param(block_path, "Unit", unit);
    elseif isfield(parameters, "OutputSignalUnit")
        set_param(block_path, "OutputSignalUnit", unit);
    else
        error("Output-unit parameter was not found on %s.", block_path);
    end
end

function add_annotation(model_name, text, position)
    annotation = Simulink.Annotation(model_name, text);
    annotation.Position = position;
    annotation.FontSize = 12;
end

function style_axes(axes_handle, neutral)
    grid(axes_handle, "on");
    axes_handle.FontName = "Times New Roman";
    axes_handle.FontSize = 11;
    axes_handle.LineWidth = 0.8;
    axes_handle.XColor = neutral;
    axes_handle.YColor = neutral;
    axes_handle.Box = "on";
end

function export_model_diagram(model_name, filename)
    try
        open_system(model_name);
        set_param(model_name, "Location", [100, 100, 1500, 850]);
        set_param(model_name, "ZoomFactor", "FitSystem");
        set_param(model_name, "PaperPositionMode", "auto");
        system_option = "-s" + model_name;
        print(char(system_option), char(filename), "-dpng", "-r300");
    catch export_error
        warning("Model diagram export failed for %s. %s", ...
            model_name, export_error.message);
    end
end

function close_if_loaded(model_name)
    if bdIsLoaded(model_name)
        close_system(model_name, false);
    end
end

function delete_if_present(filename)
    if isfile(filename)
        delete(filename);
    end
end
```